

Event-triggered active inference for online gait adaptation to sudden payload shifts in quadrupedal locomotion

Alex Pitoiset¹, Tim N. Nisslbeck¹, Ajith Anil Meera¹ and Wouter M. Kouw¹

Abstract—Central pattern generators (CPGs) produce efficient rhythmic locomotion from a few parameters, but a set tuned for one operating condition is no longer optimal once that condition changes, even with a reactive attitude controller holding the trunk level. We augment a coupled-Hopf-oscillator CPG with an attitude virtual model controller and study online re-tuning of its gait parameters. We separate *when* to change the gait – a prediction-error trigger that fires when predicted outputs diverge from a goal prior – from *what* to change it to – an active inference agent that couples a fast belief over the body’s dynamics with a learned map from gait parameters to body outcomes, selecting gaits by minimizing an expected free energy under that prior while remembering which gaits have led to a fall. Screening each candidate against this memory rather than physically trialing it, as a grid search or Bayesian optimizer must, the agent recovers online without destabilizing trials: on a shifting trunk payload recurring through a continuous bout, it learns within the bout to stop selecting the gaits that fall. How far this recovery generalizes, and how much the gait parameterization caps it, remain open.

I. INTRODUCTION

Legged machines are increasingly asked to work through long, uninterrupted bouts, across which conditions seldom hold still: the ground changes underfoot, a carried load shifts, an actuator wears down. Each such event can turn a stably walking controller into one that stumbles, so the ability to keep going through it – an adaptability animals display effortlessly – separates a laboratory demonstration from a deployable machine. Engineers have long pursued it by replicating animal morphologies [12], [5], yet the controllers rarely inherit the biological advantage: Boston Dynamics’ Spot relies on centralized, sensor-intensive controllers that are effective but complex and costly [5].

Central pattern generators (CPGs) offer a simpler alternative. In vertebrates, these spinal circuits generate the periodic signals of movements such as walking, freeing higher centers to modulate speed and gait through simple inputs while sensory feedback keeps the rhythm coupled to the body [7], [4]. This decentralized architecture lowers command latency and bandwidth [8], making CPG controllers simpler, cheaper, and less energy intensive. The price is that the rhythm is shaped by only a handful of parameters held fixed during walking, and a setting stable under one condition need not be under another; when an event changes the condition mid-bout, staying upright requires re-tuning those parameters

online. This paper studies when that re-tuning should be triggered and how the new parameters should be chosen without destabilizing the robot.

Among CPG models, coupled nonlinear oscillators are biologically plausible and computationally efficient [2], [8]: mutual inhibition synchronizes the joints into coordinated gaits [2], [15], [14], and their limit cycles recover quickly from perturbations [15] under only a few parameters. An open-loop CPG walks stably on flat ground, but a change in condition induces body rotations the rhythm alone cannot correct, so feedback closes the loop: contact feedback resynchronizes the oscillators to the actual footfalls [14], and posture feedback corrects the trunk attitude. Virtual model control (VMC) computes joint actions as if virtual springs and dampers were attached to the body [13]; Ajallooeian et al. use such springs to pull the trunk level, traversing terrain that defeats an open-loop CPG [1]. We adopt this attitude VMC as a reactive posture layer (Section II-C) and ask a complementary question: given it, when and how should the CPG’s gait parameters be re-tuned once the condition changes?

Reactive feedback leaves the parameters untouched; a large or persistent change requires adapting them. Cully et al. let a damaged robot search a precomputed behavioral repertoire by trial and error, recovering within roughly a dozen physical trials [3], while Kumar et al. infer a latent encoding of the condition from recent proprioception and adjust a learned policy within a fraction of a second [10]. Each incurs a cost we avoid: trial-and-error executes each candidate on the already-compromised robot, and the learned policy needs extensive training and is opaque. A complementary line instead tunes gait parameters as online black-box optimization: Bayesian optimization fits a Gaussian-process surrogate to the parameter-score pairs seen so far and proposes the next window’s parameters, a trust region keeping proposals from jumping to destabilizing gaits [6] – but it too can only score a candidate by walking on it. We instead adapt the few interpretable CPG parameters online and model-based, screening each candidate against a memory of past failures.

We contribute a prediction-error trigger that decides *when* to re-tune the CPG and a unified active inference agent that decides *what* to re-tune to, screening candidates against a memory of failed gaits to avoid the destabilizing on-robot trials that grid search, Bayesian optimization, and extremum-seeking control require. An ablation attributes the reduction in falls to that screening rather than to the unified objective, which matches it while removing the need for a separately

*This work was supported by the Eindhoven Artificial Intelligence Systems Institute and NGF AiNed XS Europe grant.

¹Department of Electrical Engineering, Eindhoven University of Technology, Netherlands w.m.kouw@tue.nl

tuned detector.

II. PROBLEM STATEMENT

We first describe the base platform whose motion defines the locomotion objective, then the controller driving it: a central pattern generator producing the coordinated leg motion, augmented with an attitude virtual model controller that reactively corrects posture. We then argue that, even with this reactive loop, the parameters yielding stable locomotion depend on the *operating condition* – the combined state of the environment and the robot’s own body – so a set tuned for one condition degrades once that condition changes, whether externally, as with a different terrain, or intrinsically, as with an added payload or a weakened actuator. Resuming stable locomotion after such a shift thus requires adapting the parameters online.

A. Base platform and locomotion objective

The robot is a floating base actuated through its legs, its trunk described by the base state

$$\xi = (p, \Psi, v, \omega), \quad (1)$$

collecting the linear position $p \in \mathbb{R}^3$, orientation $\Psi \in SO(3)$, linear velocity $v = \dot{p}$, and angular velocity ω of the trunk in a world frame. The orientation is parameterized by roll, pitch, and yaw (ψ_r, ψ_p, ψ_y) , of which roll and pitch measure the deviation from upright.

For a given base motion there is a corresponding configuration of the twelve joints, their rates, and the torques that realize it,

$$(\varphi, \dot{\varphi}, \tau) = f(\xi), \quad (2)$$

where φ are the joint angles, $\dot{\varphi}$ their rates, and τ the torques. The map f is unknown: it depends on the full rigid-body dynamics, the terrain, and the switching foot–ground contact, none available in closed form.

Locomotion is then the problem of finding torques τ that drive the base toward a desired behavior: holding a target forward velocity v^* with roll and pitch near zero,

$$\tau^* = \arg \min_{\tau} \|v - v^*\| + \|(\psi_r, \psi_p)\|. \quad (3)$$

Because f is unknown these torques cannot be solved for directly. Instead a central pattern generator produces rhythmic joint-angle references, an attitude virtual model controller adjusts them to counter posture deviations, and joint-level position controllers track the result through torques. We introduce both next.

B. Central Pattern Generator

The CPG is a network of four mutually coupled Hopf oscillators, one per limb – left front (LF), right front (RF), left hind (LH), right hind (RH) – coordinated by inhibitory coupling through a gait-specific matrix $K = [k_{ij}]$. Each oscillator balances stance and swing frequencies through

$$\omega_i = \omega_{\text{stance}} \zeta(-by_i) + \omega_{\text{swing}} \zeta(by_i), \quad (4)$$

with $\zeta(\cdot)$ a sigmoid interpolating between the two frequencies by the sign and magnitude of the phase y_i , its sharpness set by b , and evolves as

$$\dot{x}_i = \alpha(u - r_i^2)x_i - \omega_i y_i, \quad (5)$$

$$\dot{y}_i = \beta(u - r_i^2)y_i + \omega_i x_i + \gamma \sum_{j=1}^4 k_{ij} y_j + s_i, \quad (6)$$

where $r_i = \sqrt{x_i^2 + y_i^2}$ is the instantaneous amplitude, u the squared limit-cycle amplitude, and $\alpha, \beta > 0$ convergence rates (typically $\beta \gg \alpha$ for stronger convergence along the y -axis). The coupling gain γ strengthens inter-limb coordination and reduces the dependence on initial conditions.

The term s_i closes a feedback loop on foot-contact timing [14]. Classifying leg i as swinging above a small positive threshold on y_i and in stance below a small negative one (with a hysteresis band between, where $s_i = 0$), and comparing this phase against the binary contact state of foot i , yields

$$s_i = \begin{cases} K_{\text{stop}}(\omega_i x_i - \gamma \sum_j k_{ij} y_j) & \text{contact matches phase} \\ F_{\text{fast}} \text{sign}(y_i) & \text{otherwise} \end{cases} \quad (7)$$

The phase matches when the foot is airborne in swing or loaded in stance, and is contradicted by an early touchdown or a lost contact; the gains K_{stop} and F_{fast} set how strongly the sensed footfalls resynchronize the rhythm to the actual contacts.

The oscillator states map to joint-angle references,

$$\varphi_i^{\text{hip}} = \varphi_0^{\text{hip}} + A_{\text{hip}} x_i, \quad (8)$$

$$\varphi_i^{\text{knee}} = \varphi_0^{\text{knee}} - A_{\text{knee}} \max(0, y_i) + \Delta_i, \quad (9)$$

with standing-pose offsets $\varphi_0^{\text{hip}}, \varphi_0^{\text{knee}}$ and swing amplitudes $A_{\text{hip}}, A_{\text{knee}}$: during swing ($y_i > 0$) the knee flexes to lift the foot while the hip advances with x_i , and during stance the knee holds its nominal extension. The term Δ_i is the posture correction from the virtual model controller of Section II-C, zero for the open-loop CPG. Joint-level position controllers track these references, producing the torques τ of Eq. 2.

The CPG’s behavior is governed by a small parameter vector

$$\theta = [\gamma, \omega_{\text{swing}}, \omega_{\text{stance}}, F_{\text{fast}}, K_{\text{stop}}, A_{\text{hip}}, A_{\text{knee}}, b], \quad (10)$$

the coupling gain, the stance and swing frequencies, the contact-feedback gains F_{fast} and K_{stop} , the hip and knee amplitudes, and the sigmoid sharpness b . These set the gait shape and are held fixed during locomotion.

C. Attitude Virtual Model Control

The contact feedback of Eq. 7 synchronizes the rhythm to the footfalls, but nothing so far senses the trunk attitude – yet roll and pitch deviations are precisely what drive the robot toward a fall (Eq. 3): on uneven or sloped ground the rhythm induces body rotations it cannot itself correct. Virtual Model Control (VMC) imagines virtual springs and dampers attached to the robot and computes the joint actions they

would produce [13]. Ajallooeian et al. [1] span such springs between a trunk-fixed plane and a horizontal reference, so the virtual forces at the stance feet continually pull the trunk level.

We adopt a kinematic form suited to position-controlled joints. A virtual spring-damper on each attitude angle produces the correction commands

$$c_r = -(\kappa_p^r \psi_r + \kappa_d^r \dot{\psi}_r), \quad (11)$$

$$c_p = -(\kappa_p^p (\psi_p - \bar{\psi}_p) + \kappa_d^p \dot{\psi}_p), \quad (12)$$

with stiffness and damping gains κ . Roll is regulated toward zero, since straight walking involves no sustained bank and lateral tip-over is the dominant fall mode; pitch is regulated toward a slowly adapting baseline $\bar{\psi}_p$ – an exponential moving average of recent pitch, time constant ~ 1 s – so transient rocking is damped while the sustained pitch of an incline is tolerated rather than fought. The commands distribute over the legs as the knee corrections of Eq. 9,

$$\Delta_i = \text{clip}(\sigma_i^F c_p + \sigma_i^L c_r, \pm \delta_{\max}), \quad (13)$$

with $\sigma_i^F = \pm 1$ for front/hind legs, $\sigma_i^L = \pm 1$ for left/right legs, and $\delta_{\max}$ bounding the correction. Since a knee angle sets the effective leg length, the legs on a sinking side of the trunk extend to push that corner up while those on the rising side shorten, realizing the virtual springs of Ajallooeian et al. [1] through the position-controlled knees.

The VMC gains are fixed, hand-tuned, and not part of θ . The attitude loop thus acts within a stride, correcting momentary deviations but leaving the rhythm itself untouched – and Ajallooeian et al. [1] observe that these corrections lose effectiveness on harder terrain, where they begin to work against the CPG’s forward progression. Whether locomotion stays stable therefore still hinges on θ matching the operating condition, which we examine next.

D. Shifts in Parameter Optimality

Let e denote the *operating condition*: the combined state of environment and body the gait must accommodate – extrinsic factors such as terrain incline, friction, and micro-geometry, together with intrinsic ones such as the trunk’s mass distribution and each actuator’s torque capacity. Running the controller with parameters θ under condition e , with the attitude feedback of Section II-C active, induces a distribution $p(\xi | \theta, e)$ over the base state, which we score by the locomotion objective of Eq. 3,

$$V(\theta, e) = -\mathbb{E}_{p(\xi | \theta, e)} \left[\frac{1}{v^*} \|v - v^*\| + \frac{1}{\psi_0} \|(\psi_r, \psi_p)\| \right], \quad (14)$$

setting $V = -c_f$ on a fall. The scale ψ_0 lets the attitude term dominate, encoding uprightness as the primary requirement; in practice the expectation is a time average over an evaluation window on low-pass-filtered angles, so the gait’s rhythmic rocking is not penalized. The best gait for condition e is the condition-conditioned optimum

$$\theta^*(e) = \arg \max_{\theta} V(\theta, e). \quad (15)$$

Crucially, $\theta^*(e)$ is not constant: optimizing Eq. 14 under differing terrain, load, or actuator torque yields distinct parameter vectors, each most stable under the condition it was tuned for, so a change in condition relocates the optimum,

$$\theta^*(e_{\text{old}}) \neq \theta^*(e_{\text{new}}), \quad (16)$$

a *shift in parameter optimality*. Such shifts arise externally, as when the robot steps onto a new surface, and intrinsically, as when it picks up a payload or an actuator loses torque, and may drift slowly or jump at a discrete *event*. The experiments of Section IV study one abrupt, intrinsic event – a shifting payload – that relocates the optimum too sharply for the reactive posture loop alone to absorb.

Because θ is held fixed, a shift leaves the controller suboptimal and the attitude deviations compound: as the trunk tilts, recovery soon demands more than the bounded feedback of Eq. 13 can supply, and locomotion fails. Resuming stable walking therefore requires two decisions – *when* the optimum has moved, and *what* to re-tune θ toward – made online, quickly enough to preserve balance. The next section addresses both with a single active inference agent, in which one goal prior encoding the objective of Eq. 3 both detects the shift and selects the new gait.

III. METHODS

A. Unified active inference agent

The controller is a single active inference agent organized around one Gaussian *goal prior* over the body outputs $\mathbf{y}_k = (v_x, v_y, \phi, \rho)_k$ — forward and lateral trunk velocity, pitch and roll — namely $\bar{p}(\mathbf{y}) = \mathcal{N}(\mathbf{y}^*, \Sigma^*)$ with $\mathbf{y}^* = (v^*, 0, 0, 0)$: walk forward at the target speed, level and straight. Two probabilistic models share this prior.

A *dynamics belief*. A multivariate autoregressive model with exogenous input (MARX) predicts the outputs from their recent past and the measured joint angles $\mathbf{q}_k$,

$$\mathbf{y}_k = \sum_{i=1}^p A_i \mathbf{y}_{k-i} + B \mathbf{q}_k + \mathbf{e}_k, \quad (17)$$

the exogenous term $B \mathbf{q}_k$ a linear proprioceptive map from joint configuration to body motion. A matrix-normal–Wishart posterior over the coefficients $(A_{1:p}, B)$ and noise precision is updated in closed form at every control step (100 Hz) [11], giving a Gaussian one-step posterior predictive $\mathcal{N}(\mu_k, \Sigma_k)$.

A *control map*. A Gaussian process maps the CPG parameters θ to the outcome outputs, fitted to a persistent memory $\{(\theta_j, \mathbf{y}_j)\}$ of the gaits tried and the outputs they produced (a fall recorded as its degraded output). Only the high-leverage subset of the parameters of Eq. 10 is searched, the rest held at the incumbent, and the memory persists across recurrences, so the map fills in as the condition repeats and no gait is supplied in advance.

When adaptation is triggered (Section III-B), the agent proposes the gait minimizing the expected free energy of its

predicted output under the goal prior,

$$G(\theta) = \underbrace{\frac{1}{2}(\mu_\theta - \mathbf{y}^\star)^\top \Sigma^{\star-1}(\mu_\theta - \mathbf{y}^\star) + \frac{1}{2}\text{tr}(\Sigma^{\star-1}\Sigma_\theta)}_{\text{goal seeking (cross-entropy to goal)}} - \underbrace{\frac{1}{2}\ln|I + \Sigma_\theta \Sigma_n^{-1}|}_{\text{information gain}}, \quad (18)$$

where $(\mu_\theta, \Sigma_\theta)$ is the GP posterior predictive of the output at θ and Σ_n the observation noise, and it proposes $\hat{\theta} = \arg\min_\theta G(\theta)$ over a quasi-random pool of candidates. Safety is not a separate constraint but a consequence of the goal prior: a gait the memory associates with falling predicts outputs far from $\mathbf{y}^\star$, inflating the goal-seeking term, so it is avoided while the agent still explores gaits it has not yet ruled out. The performance of any gait is scored as the log-likelihood of its observed output under $\bar{p}$. Because the agent screens each candidate against its own map before committing, it recovers a gait without the destabilizing on-robot trials a black-box optimizer needs to score a candidate, the property Section IV turns on.

The two models are coupled rather than stacked: they share one goal prior — the dynamics belief drives the trigger on the prior’s attitude marginal (Section III-B) and the control map proposes the gait under the full prior — so sensing, triggering and control minimize one free energy.

B. Prediction-error triggering

Re-tuning θ at every step is wasteful and, as Section IV shows, actively destabilizing, so the agent adapts only when its own predictions leave the goal. The perturbations the agent must survive threaten the trunk’s *uprightness*, not its speed: a shift a slower gait can absorb is tolerable, whereas a tilting trunk is not. The trigger therefore reads the attitude marginal of the goal prior — the cross-entropy of the dynamics belief’s one-step predicted pitch and roll $(\mu_k^\rho, \Sigma_k^\rho)$ to their level target,

$$\varepsilon_k = H(\mathcal{N}(\mu_k^\rho, \Sigma_k^\rho) \parallel \bar{p}^\rho) \quad (19)$$

$$= \frac{1}{2}\mu_k^{\rho\top} \Sigma_\rho^{\star-1} \mu_k^\rho + \frac{1}{2}\text{tr}(\Sigma_\rho^{\star-1} \Sigma_k^\rho) + c, \quad (20)$$

the same cross-entropy-to-goal that Eq. 18 minimizes, restricted to the attitude subspace (the level target has zero mean) and evaluated on the belief rather than a candidate gait: it is small while the trunk is predicted level and grows when a perturbation drives it to tilt, requiring no model of the perturbation and no signal of when it occurs. Standardizing ε_k against a healthy warm-up baseline $(\bar{\varepsilon}, s)$ and accumulating the excess in a one-sided CUSUM,

$$S_k = \max(0, S_{k-1} + (\varepsilon_k - \bar{\varepsilon})/s - \kappa), \quad (21)$$

fires an event when S_k exceeds a bound h ; the slack κ is the per-step surprise tolerated as noise. Because the CUSUM integrates over time, it registers the sustained mismatch a perturbation creates while rejecting the stride-frequency ripple that dominates a one-step residual at 100 Hz.

The monitor is armed only after a sustained healthy interval past a grace period, so the transient following a

recovery is never mistaken for a fresh perturbation. Once a gait is applied the accumulator is reset and held at zero for a settle window covering the ramp-in and one scoring hold, so the ramp transient cannot itself re-trigger; the monitor then latches for the remainder of the event, and a single perturbation therefore draws a single correction. Re-tuning runs only while the trunk is measurably off-level. Reading uprightness alone is what lets the trigger *quiet*: once the robot is upright again it falls silent even under a recovered gait that walks slower or must crab sideways to absorb an asymmetric fault, so the agent stops adapting when it has restored balance rather than chasing a nominal speed it may no longer reach. The forward-velocity preference is not discarded — it stays in the control objective of Eq. 18, which keeps seeking speed among the gaits that hold the trunk level. Sensing and control thus read one goal prior two ways: its attitude marginal decides *when* to act, the full prior decides *what* to do.

C. Setting the threshold from the cost of adapting

The bound h in Eq. 21 need not be tuned, because the accumulator carries units. Since ε_k is a cross-entropy in nats, sS_k is the free energy forgone, in nats, by holding the incumbent gait since the excursion began, so h is a *price* rather than a sensitivity. Adapting carries a price of its own: writing $p(\theta) = \mathcal{N}(\theta_k, \Upsilon^{-1})$ for a prior over gait parameters centered on the incumbent, a switch to $\theta_k + \Delta\theta$ costs

$$C(\Delta\theta) = -\ln p(\theta_k + \Delta\theta) + c = \frac{1}{2}\Delta\theta^\top \Upsilon \Delta\theta, \quad (22)$$

paid once at the switch, whereas ε_k is a rate incurred afresh every step the incumbent is held; a threshold on ε_k itself thus compares a rate to a one-time cost, and is either tripped by transients or, set honestly, silent until the robot is already failing. Accumulating first removes the inconsistency, and the bound becomes the price,

$$h = C(\Delta\theta)/s, \quad (23)$$

turning Eq. 21 into a payback test: the agent switches once the value forgone by not switching repays what switching costs, after a delay $h/(\bar{\varepsilon} - \kappa)$ that shortens as the standardized excess $\bar{\varepsilon}$ grows. The slack κ keeps its usual role of fixing the in-control average run length and is the only quantity still calibrated by hand.

Two caveats limit how literally Eq. 23 may be read: it prices only the subspace the gait parameters can steer, so it is a *lower* bound on the principled threshold, and it is only as meaningful as Υ . At the operating point of Section IV-A, $h = 5$ prices a switch at 0.13 nats under the measured $s = 0.025$ nats — a fifth of a prior standard deviation — whereas the gaits the agent actually applies lie some twelve standard deviations out; pricing that realized move would delay the trigger by a median of 10.8 s and silence it in most bouts. The discrepancy reflects that the realized move is large because the search explores, not because correction demands it; pricing the correction the belief itself deems necessary, via its control gain, would close this gap, which we leave to future work. Calibration in the experiments below therefore rests on κ : over the 150 s of settled nominal walking the

standardized excess never exceeds 2.7, so at $\kappa = 3$ every increment is negative, the accumulator cannot leave zero, and no false alarm is possible whatever h is; h only imposes a short persistence requirement once a perturbation lifts the excess past the slack. (The transient following a recovery reaches several hundred standard deviations and is instead kept from firing by the arming logic of Section III-B.)

What the slack buys can be stated precisely. Writing $e_k = (\epsilon_k - \bar{\epsilon})/s$ and $T = \inf\{k : S_k > h\}$ for the firing time, the accumulator admits both a deterministic and an exponential guarantee against false alarms.

Proposition 1. *Conditional on the warm-up estimates $(\bar{\epsilon}, s)$, the recursion of Eq. 21 satisfies: (i) if $e_k \leq \kappa$ for every $k \leq n$, then $S_k = 0$ for every $k \leq n$ and $T > n$, for any bound $h > 0$; (ii) if the increments $e_k - \kappa$ are independent and identically distributed with negative mean, and some $\theta^* > 0$ solves $\mathbb{E}[e^{\theta^*(e_k - \kappa)}] = 1$, then the trigger fires during an unperturbed run of unbounded length with probability at most*

$$\Pr(T < \infty) \leq e^{-\theta^* h}. \quad (24)$$

A proof sketch is given in the Appendix. Part (i) governs the experiments: the observed excess peaks at 2.69 over the 150 s baseline, below $\kappa = 3$, so no false alarm is possible on that sample regardless of h . Part (ii) covers excursions beyond the observed support: for Gaussian e_k , $\theta^* = 2(\kappa - \mu)/\sigma^2$ gives $e^{-2\kappa h} = 9 \times 10^{-14}$ at the standardized values, though this is optimistic since ϵ_k is strongly autocorrelated at 100 Hz (lag-one correlation 0.92, long-run variance 8.9 $\times$ the marginal); substituting the long-run variance for the one-step generating function instead gives $\theta^* \approx 2.7$ and a still-serviceable bound below 10^{-6} . Both figures are conditional on $(\bar{\epsilon}, s)$, themselves estimated from a 125-sample warm-up with an integrated autocorrelation time of about 33, i.e. roughly four effective samples.

No matching guarantee is claimed against missed detections: it would require the post-change increments to have positive mean, whereas the payload shift of Section IV-A raises the mean standardized excess only to 0.93 against $\kappa = 3$, so the accumulator still drifts downward on average and fires through excursions of a heavy-tailed statistic rather than through drift. This suffices in practice – every event in Section IV-A is detected – but we treat the false-negative rate as an empirical matter throughout.

IV. EXPERIMENTS

We evaluate the unified active inference agent on the Laikago quadruped in PyBullet, on a perturbation of the robot’s own dynamics: a shifting trunk payload (Section IV-A). The agent runs with the attitude VMC of Section II-C active and adapts only the high-leverage subset of the parameter vector θ of Eq. 10, within fixed bounds throughout, while adaptation is gated by the prediction-error trigger of Section III-B. Holding the incumbent gait θ^* (flat) without adapting is the reference against which the agent is measured. Control runs at $\Delta t = 0.01$ s with a target forward velocity of

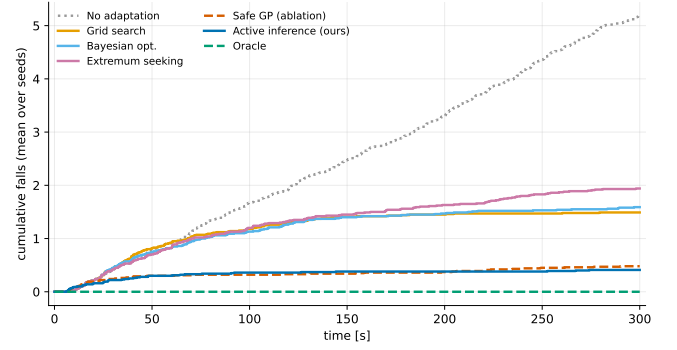


Fig. 1. Cumulative falls over a 300 s continual bout under the persistent payload shift (mean over one hundred seeds). Holding the incumbent gait accrues falls throughout. The three arms that must trial a candidate on the loaded robot (grid search, Bayesian optimization, extremum seeking) each find a tolerant gait early and then largely stop falling, but pay for the search first. The two arms that screen candidates against a memory of past falls, our agent and the safe Gaussian-process ablation, fall far less and are indistinguishable from each other, which locates the benefit in the screening rather than in the unified objective. A clairvoyant oracle never falls.

0.5 m/s; each perturbation event is scored by the criterion of Eq. 14 over a hold window with the ramp-in excluded, and gait switches are ramped in over 0.3 s so the gait never jumps.

Adaptation runs asynchronously with the simulation, as it would on hardware: the controller steps the physics at 100 Hz on one thread while each gait proposal — the expected-free-energy minimization of Eq. 18, its Gaussian-process fit included — is computed concurrently off the control loop, so the simulation never pauses to wait for it. A proposed gait takes effect only once the computation returns; the optimizer’s wall-clock cost therefore appears as a control latency, over which the robot keeps walking on the unadapted gait, rather than as a frozen clock. Because each bout is paced to real time, a slower optimizer is charged honestly — through the extra ground covered before its gait engages — and we report this compute latency alongside the detection latency below.

A. Adapting to payload shifts

The second scenario perturbs the robot’s mass distribution rather than its actuation. The Laikago carries an 8 kg payload rigidly attached above its trunk, whose own mass is 10 kg, and at random intervals the payload shifts off the sagittal plane, 0.215 m laterally and 0.20 m to the rear, ramped over 1 s so the change is a sustained bias rather than an impulse. The offset then persists: once engaged it is held indefinitely, so the trunk bias compounds until the robot either falls or settles into a gait that tolerates the load. This offset sits at the edge of recoverability: holding the incumbent gait θ^* (flat) the robot tips over on 85% of shifts, after a median of 33 s under the load, yet a re-tuned gait can stay upright indefinitely, so the perturbation rewards adaptation without being hopeless. The bout is non-episodic and runs for 300 s: whenever the robot falls it is stood upright and the payload recentered, and after 2 to 8 s the shift recurs, so a controller

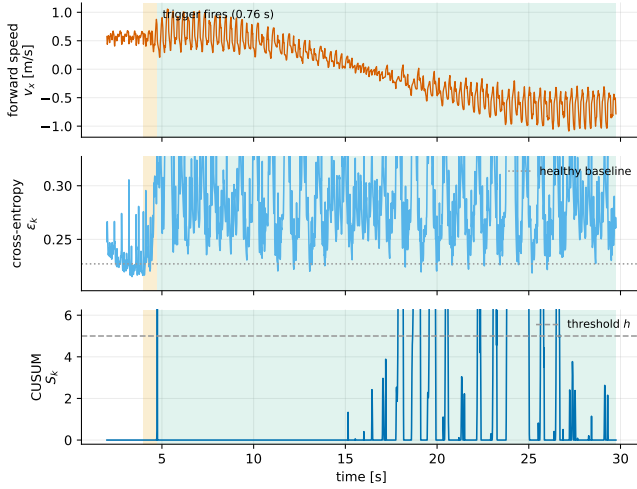


Fig. 2. The uprightness-only trigger over one payload shift (ours). *Top*: forward speed, which the tolerant gait carries slowly backward under the load. *Middle*: the attitude cross-entropy ϵ_k rests at its healthy baseline and rises when the shift tilts the trunk (shaded orange until the trigger fires, green once the proposed gait is active); it settles above baseline rather than returning to it. *Bottom*: the CUSUM S_k crosses its threshold h within 0.76 s, is held at zero through the settle window, and re-crosses later in the bout without drawing a second adaptation, the monitor having latched for the event.

that never adapts meets the same perturbation over and over, whereas one that finds a tolerant gait simply holds it and walks on. An agent that remembers earlier outcomes should therefore stop repeating the parameter choices that fell.

We evaluate the unified active inference agent of Section III-A. At each detected shift it proposes a gait by minimizing the expected free energy of Eq. 18 over its control map, favoring parameters whose predicted outputs lie near the goal while avoiding the regions its memory associates with falling, and here it searches only the high-leverage subset $\{\gamma, \omega_{\text{swing}}, F_{\text{fast}}, K_{\text{stop}}, b\}$ of Eq. 10; the memory persists across recurrences within the trial, so the map fills in as the shift repeats. Adaptation is triggered by the cross-entropy criterion of Section III-B: the MARX dynamics belief, updated online from the trunk outputs and the measured joint angles, drives the CUSUM once its one-step prediction departs the goal, re-arming once the robot has recovered after each recenter.

Five further arms, each searching that same subset so the comparison is head to head, place the agent among the alternatives. Three are online optimizers that can judge a candidate gait only by running it: Latin-hypercube sampling (grid search), online Bayesian optimization, and extremum-seeking control, the classical model-free tuner, which dithers each parameter at a distinct frequency and demodulates the event score into a gradient estimate [9]. Extremum seeking is the sharpest form of the difficulty, since even its exploratory dither must be executed on the already-loaded robot. The fourth is an ablation that isolates the active inference machinery from safety-filtered search: a Gaussian-process agent that screens candidates against the same memory of past falls, but driven by a hand-tuned prediction-error trigger rather

than by the attitude marginal of the goal prior, and carrying no MARX belief of the body’s dynamics. The fifth is a clairvoyant oracle that jumps straight to a pre-fit tolerant gait, an upper anchor on any parameter switch.

Because a tolerant gait, once found, simply keeps walking, we run one hundred 300 s bouts per method and report the *cumulative* falls each accrues over a bout rather than a per-event rate (Fig. 1). Holding the incumbent, the robot falls 5.18 ± 0.18 times per bout (mean $\pm$ s.e.m.): it never adapts and tips over on 85% of recurrences. Our agent instead falls only 0.41 ± 0.08 times, finding a tolerant gait after fewer than one fall on average and within the first 20 s, after which it holds that gait for the rest of the bout. The three online optimizers fall in between, at 1.49 ± 0.08 (grid search), 1.59 ± 0.09 (Bayesian optimization), and 1.94 ± 0.05 (extremum seeking) falls per bout, each significantly worse than our agent (Mann-Whitney on per-seed fall counts, $p < 10^{-26}$). They fall more because each can assess a candidate only by executing it on the already-shifted robot, where a poor candidate tips it over rather than merely scoring badly. Extremum seeking is worst of the three, significantly worse than both grid search ($p \approx 6 \times 10^{-9}$) and Bayesian optimization ($p \approx 3 \times 10^{-6}$), even though its proposal is the cheapest to compute: its dither is a perturbation applied to the loaded robot by construction, so the exploration it needs to estimate a gradient is paid for in falls. A clairvoyant oracle never falls, confirming a recoverable gait exists within the searched subset.

The ablation, however, qualifies what the active inference machinery contributes. The safety-filtered Gaussian-process agent, without the unified trigger and without the MARX belief, falls 0.48 ± 0.08 times per bout, statistically indistinguishable from our agent’s 0.41 ± 0.08 ($p = 0.5$), and its fall accrual tracks ours closely throughout the bout (Fig. 1). What separates the two groups is therefore whether a candidate is screened against a memory of past falls before it is walked on, not whether the trigger and the controller are read off a single goal prior. The unified formulation should accordingly be read as an architectural simplification, one objective serving both roles at no cost in falls, rather than as the source of the improvement. Where it does show a measurable edge is in promptness: the goal-prior trigger detects the shift in 0.93 s on average against 1.26 to 1.38 s for the prediction-error monitors driving the other arms, with no false alarms in any arm. Under the asynchronous protocol the gait proposal costs a median of one control step, as a tolerant gait is usually found before any Gaussian-process fit is needed, rising to 1.74 s in the tail once the map is being fit. The agent also stops re-tuning once a tolerant gait is in place, adapting exactly once per shift event across all one hundred bouts, so it does not chatter (Fig. 2). Two measurements qualify what produces that restraint. The attitude prediction-error does not return to its healthy baseline: it settles about 17% above the pre-shift level, within 10% of baseline on only 5% of bouts, since a load held off-center is never quite free. The accumulator does re-cross its threshold later in

most bouts, and no second adaptation follows because the monitor latches for the remainder of the event. The restraint is therefore a property of the event logic rather than evidence that the residual mismatch has vanished, and whether re-tuning on that residual would help is untested.

At this offset the tolerant gaits are more compromised than falls alone convey. Every adapting arm settles into a gait that walks slowly *backward* under the load, at -0.12 m/s for our agent and -0.14 m/s for the three optimizers, against a 0.5 m/s forward target. The clairvoyant oracle is the most retrograde of all at -0.22 m/s, which is the informative part: the pre-fit gait that never falls is itself a backward one, so within this parameter subset no gait both tolerates the offset and makes forward progress. The benefit here is thus strictly falls avoided, and reaching a gait that walks forward under the load is capped by the gait parameterization, as we discuss next.

V. DISCUSSION

Our results separate two aspects of online adaptation that are easily conflated: deciding *when* to change the gait and deciding *what* to change it to. The prediction-error trigger resolves the first cheaply, reading uprightness rather than speed and re-arming to catch a recurring sequence, without a model of the perturbation. The second is where the danger lies: grid search, Bayesian optimization, and extremum seeking can judge a candidate gait only by walking on it, so on an already-perturbed robot a poor candidate does not merely score badly, it tips the robot over – extremum seeking makes the point most sharply, its dither a deliberate perturbation of the loaded robot that costs more falls than the two search methods it is cheaper than. Our agent instead screens each candidate against a learned map with an explicit memory of past falls, avoiding these destabilizing trials by making small, model-guided corrections rather than a faster search toward a distant optimum. This holds even though the CPG is already stabilized by the attitude virtual model controller, so the benefit is complementary to the reactive posture loop rather than a substitute for it.

The ablation narrows this claim: a safety-filtered Gaussian-process agent, given the same fall memory but neither the unified goal-prior trigger nor the dynamics belief, matches our agent’s fall rate, so it is the screening that pays, not the unified objective. We therefore read the unified formulation as an architectural economy – one goal prior specifying when to act and what to do, removing a separately tuned detector at no cost in falls here and detecting the shift somewhat sooner – rather than as a performance result; whether that economy earns more where a detector’s threshold is harder to hand-tune remains untested.

Several limitations qualify these conclusions. The evaluation is in simulation, over a single intrinsic perturbation and a modest number of recurrences, and the reference optima are computed offline. More fundamentally, how much adaptation can recover is capped by the gait parameterization itself: under this payload offset no gait in the searched subset both tolerates the load and advances, the clairvoyant oracle itself

walking backward at 0.22 m/s, so every online method settles somewhere between it and a standstill. Staying upright and making progress are thus in tension here, and our agent, like the others, resolves it toward staying upright – a clean comparison since all arms pay the same price, but one that leaves adapting *without* surrendering the locomotion objective open, likely requiring a richer parameterization rather than a better search within this one. A richer control representation, or a model expressive enough to reach the distant optimum without destabilizing exploration, is the central open problem; validating on hardware, and across a wider range of perturbations, are natural next steps.

VI. CONCLUSIONS

We augmented a coupled-Hopf-oscillator CPG with an attitude virtual model controller and studied when and how to re-tune its gait parameters online as the operating condition changes. A prediction-error trigger decides when to adapt from a sustained loss of performance, and a unified active inference agent decides what to change by selecting parameters from a learned model of the body’s response, with a persistent memory of past falls, rather than by physically trialing candidates. On a shifting trunk payload, the agent learns within a single bout to stop selecting the gaits that fall, recovering without the destabilizing trials that grid search, Bayesian optimization, and extremum seeking require. An ablation attributes that gain to the memory-screened selection rather than to the unified objective, which matches it on falls while dispensing with a separately tuned detector. How far this recovery generalizes, and how much the gait parameterization itself caps it, remain open.

APPENDIX

Write $X_k = e_k - \kappa$ and $W_n = \sum_{k=1}^n X_k$ (with $W_0 = 0$), so Eq. 21 reads $S_n = \max(0, S_{n-1} + X_n)$, $S_0 = 0$. (i) If $X_k \leq 0$ for every $k \leq n$, induction gives $S_k = 0$ for every $k \leq n$ (it holds at $k = 0$, and $S_{k-1} = 0 \Rightarrow S_k = \max(0, X_k) = 0$), hence $T > n$ for any $n > 0$: the accumulator cannot leave the origin on a stretch whose excess never exceeds the slack. (ii) Unrolling the recursion gives the Lindley representation $S_n = \max_{0 \leq j \leq n} (W_n - W_j)$; since $(X_1, \dots, X_n)$ and its reversal share a distribution, S_n has the law of $\max_{0 \leq m \leq n} W_m$ for each n , so $\sup_n S_n \stackrel{d}{=} M := \sup_n W_n$ and $\{T < \infty\} = \{M > h\}$. Let $\theta^* > 0$ solve $\mathbb{E}[e^{\theta^* X_k}] = 1$. Then $Z_n = \exp(\theta^* W_n)$ is a non-negative martingale with $Z_0 = 1$; applying optional stopping at the bounded time $\tau \wedge n$, with $\tau = \inf\{n : W_n > h\}$, gives $1 = \mathbb{E}[Z_{\tau \wedge n}] \geq \mathbb{E}[Z_\tau \mathbf{1}\{\tau \leq n\}] \geq e^{\theta^* h} \Pr(\tau \leq n)$. Letting $n \rightarrow \infty$ and using $\Pr(\tau \leq n) \uparrow \Pr(M > h)$ yields $\Pr(T < \infty) = \Pr(M > h) \leq e^{-\theta^* h}$, Eq. 24. ■

For Gaussian increments $e_k \sim \mathcal{N}(\mu, \sigma^2)$, $\theta(\mu - \kappa) + \theta^2 \sigma^2 / 2 = 0$ gives $\theta^* = 2(\kappa - \mu) / \sigma^2$, or $\theta^* = 2\kappa$ and bound $e^{-2\kappa h}$ on standardized values. When the X_k are dependent, the same martingale argument applies to the limiting scaled cumulant generating function $\Lambda(\theta) = \lim_{n \rightarrow \infty} n^{-1} \ln \mathbb{E}[e^{\theta W_n}]$ in place of the one-step one: for stationary sequences under geometric mixing, Λ is finite near the origin and $\Pr(T <$

$\infty) \leq C e^{-\theta^* h}$ with $\Lambda(\theta^*) = 0$ and a constant $C \geq 1$ set by the mixing rate, the exponent degrading roughly with the ratio of long-run to marginal variance (the substitution used in Section III-C). Both statements are conditional on the warm-up estimates $(\bar{\epsilon}, s)$ that define e_k .

REFERENCES

- [1] Mostafa Ajallooeian, Soha Pouya, Alexander Sproewitz, and Auke J Ijspeert. Central pattern generators augmented with virtual model control for quadruped rough terrain locomotion. In *IEEE International Conference on Robotics and Automation*, pages 3321–3328. IEEE, 2013.
- [2] J. Collins and S. A. Richmond. Hard-wired central pattern generators for quadrupedal locomotion. *Biological Cybernetics*, 71(5):375–385, 1994.
- [3] Antoine Cully, Jeff Clune, Danesh Tarapore, and Jean-Baptiste Mouret. Robots that can adapt like animals. *Nature*, 521(7553):503–507, 2015.
- [4] Michael H. Dickinson, Claire T. Farley, Robert J. Full, M. A. R. Koehl, Rodger Kram, and Steven Lehman. How animals move: An integrative view. *Science*, 288(5463):100–106, 2000.
- [5] Rohit Dwivedula, Sadanand Modak, Aditya Akella, Joydeep Biswas, Daehyeok Kim, and Christopher J. Rossbach. Configbot: Adaptive resource allocation for robot applications in dynamic environments. *arXiv preprint arXiv:2501.10513*, 2025.
- [6] David Eriksson, Michael Pearce, Jacob Gardner, Ryan D Turner, and Matthias Poloczek. Scalable global optimization via local Bayesian optimization. *Advances in Neural Information Processing Systems*, 32, 2019.
- [7] Martyn Goulding. Circuits controlling vertebrate locomotion: moving in a new direction. *Nature Reviews Neuroscience*, 10(7):507–518, 2009.
- [8] Auke Jan Ijspeert. Special issue: Central pattern generators for locomotion control in animals and robots: A review. *Neural Networks*, 21(4):642–653, 2008.
- [9] Nick J Killingsworth and Miroslav Krstic. PID tuning using extremum seeking: online, model-free performance optimization. *IEEE Control Systems Magazine*, 26(1):70–79, 2006.
- [10] Ashish Kumar, Zipeng Fu, Deepak Pathak, and Jitendra Malik. RMA: Rapid motor adaptation for legged robots. In *Robotics: Science and Systems*, 2021.
- [11] Ajith Anil Meera and Wouter M Kouw. Curvature-aware expected free energy as an acquisition function for Bayesian optimization. *IEEE Control Systems Letters*, 2026.
- [12] Kristin Y. Pettersen. Snake robots. *Annual Reviews in Control*, 44:19–44, 2017.
- [13] Jerry Pratt, Chee-Meng Chew, Ann Torres, Peter Dilworth, and Gill Pratt. Virtual model control: An intuitive approach for bipedal locomotion. *International Journal of Robotics Research*, 20(2):129–143, 2001.
- [14] Ludovic Righetti and Auke Jan Ijspeert. Pattern generators with sensory feedback for the control of quadruped locomotion. In *IEEE International Conference on Robotics and Automation*, pages 819–824, 2008.
- [15] Alexander Sprowitz, Ludovic Righetti, and Auke Jan Ijspeert. Passive compliant quadruped robot using central pattern generators for locomotion control. In *IEEE RAS & EMBS International Conference on Biomedical Robotics and Biomechatronics*, pages 710–715, 2008.